Diagnostic Skill Reuse for Agentic Autotuning of Stateful PD-Disaggregated Serving in SGLang

BoTao Cheng
Tencent

Abstract

Configuring prefill–decode (PD) disaggregated language model serving requires decisions about phase allocation, parallelism, batching, and state transfer. Historical runs on a P800 pool indicate that nominally similar configurations can exhibit different performance and failure behavior under changed cache, initialization, and execution conditions. We analyze 35 author-run records to motivate an experience-constrained diagnostic-skill design for agentic autotuning in SGLang. A skill specifies applicability, bounded actions, controlled variables, alternative expected observations, restoration, and evidence provenance. A language-model controller is designed to select diagnostics and parameter families, while a constrained numerical optimizer selects concrete configurations. Capability and execution checks restrict admissible actions; historical experience supplies context-dependent guidance that requires local replay when its conditions change. Diagnostic support, process survival, request completion, and SLO attainment are recorded separately. We formulate resource and tuning-cost objectives and specify matched comparisons that isolate numerical search, agent decisions, and skill reuse. Historical diagnosis was human-directed; autonomous selection and its performance advantage have not yet been evaluated. The present evidence establishes motivation and an execution foundation, not agent superiority.

Keywords: LLM serving; prefill–decode disaggregation; agentic autotuning; diagnostic skills; resource planning.

1 Introduction

LLM serving systems expose configuration choices across resource allocation, parallel execution, memory management, batching, and communication. Prefill–decode (PD) disaggregation makes phase-specific choices possible, but also introduces explicit transfer of request state. DistServe studies phase allocation under latency constraints, Splitwise examines phase-specific hardware provisioning, and HexGen-2 jointly considers heterogeneous resources and communication [7, 16, 23]. These results establish that the stages and their connections should be considered together. They do not make PD deployment itself a new contribution of the present work.

This manuscript focuses on a decision that occurs before choosing a new configuration: *which observation or controlled experiment should be obtained next?* Consider a deployment whose service objective is no longer met after the request mix changes. Adding a prefill instance, reducing decode admission, and changing an available transfer setting affect different parts of the execution. The same aggregate latency symptom may be consistent with several explanations. A tuner can search all choices numerically, but may spend substantial time redeploying configurations that do not address the current limitation. An agent can interpret heterogeneous evidence, but a plausible explanation alone does not validate a diagnosis.

The use of LLMs for tuning is established. GPTuner combines domain knowledge and numerical optimization, while STELLAR analyzes logs and accumulates contextual tuning rules. SysInsight links system internals, diagnosis, and empirically supported rules [5, 9, 21]. Accordingly, adding tool calls, contextual memory, or rule confidence is not sufficient novelty. Our narrower research

question is whether an explicit *diagnostic experiment and its local replay conditions* provide useful decision structure for PD serving, where a deployment change can alter several connected stages.

We propose to represent a diagnostic skill as a bounded program that acquires observations capable of supporting, contradicting, or leaving unresolved a set of stage-related hypotheses. A controller can use the resulting evidence to prioritize a parameter family. The skill also records where its evidence was obtained. Reuse in a changed context requires compatibility checks and may require a new probe. This design treats a diagnosis as a revisable empirical statement, not as an instruction to repeatedly apply the last successful parameter adjustment.

The intended system has one LLM controller, an authored diagnostic library, a numerical optimizer, and an independent executor and evaluator. Existing work provides human-directed diagnosis and skill execution, including a configuration-to-benchmark reporting chain. The ledger contains 34 runs on 32 cards and one on 48 cards under different conditions. The starting pool uses eight-P800 servers with dual-socket AMD EPYC hosts and RDMA; resources can be expanded. AMD denotes host CPUs. These records motivate the design but do not establish autonomous agent effectiveness.

This study develops an evidence chain from historical observations to a testable method: it identifies state and protocol sensitivity in exploratory records; formulates budgeted evidence acquisition and configuration selection; and specifies constrained diagnostic skills with local reuse checks and nonexclusive search guidance. The planned evaluation tests value beyond numerical search, a fixed-rule controller, and an ordinary agent. Historical characterization is preliminary; the method contribution remains to be established experimentally.

2 Background and Related Work

2.1 Disaggregated serving and configuration

PD serving separates the processing of prompts and the generation of subsequent tokens across model replicas. The practical configuration space includes the number of replicas, their parallel strategies, runtime limits, and supported transfer paths. Mooncake organizes service around KV state, and Sarathi-Serve provides an important colocated alternative based on chunked prefill [1, 17]. A fair evaluation must therefore consider a tuned colocated system as well as tuned native PD.

SGLang provides a language-model execution system and runtime [22]. Its current PD documentation and the SGLang-Kunlun repository provide relevant integration surfaces [2, 18]. These evolving sources do not establish that every documented feature is available in the deployed P800 image. Our design requires a version-specific capability manifest and does not assume support for unequal tensor parallelism, a particular direct-memory path, or arbitrary replica routing.

Recent preprints also address SLO-aware PD resource counts and coordinated phase/network autoscaling [8, 11]. Variable resource counts and network-aware provisioning are therefore not gaps claimed here. Our initial planner recommends a deployment offline; it does not claim the online control mechanisms of an autoscaler.

Large-scale NVIDIA and Meta studies explore deployment trade-offs with simulation supplemented by empirical measurements [14, 15]; they do not rely only on exhaustive hardware trials. A re-evaluation studies KV transfer media, reuse, and performance–energy implications, while TaiChi unifies aggregation and disaggregation under TTFT/TPOT objectives [10, 19]. Cache sensitivity and multi-objective exploration are therefore background, not sufficient novelty. Our question concerns selecting and revalidating diagnostic actions in stateful serving.

2.2 Numerical and knowledge-guided autotuning

SCOOT applies constrained Bayesian optimization to inference-engine tuning and handles known and hidden configuration constraints [4]. It is a direct algorithmic reference for numerical search.

GPTuner extracts tuning knowledge and uses it for knob selection and search-space optimization; λ-Tune generates complete database configurations and accounts for candidate-evaluation overhead [6, 9]. These works motivate comparisons against capable search and knowledge-guided methods rather than against defaults alone.

SLO-Guard studies failure-aware, budget-consistent serving tuning, while ECO combines a structured prior and constrained optimization for disaggregated serving under limited measurements [13, 24]. Both are preprints in the cited versions. Failure accounting, model-assisted search, and a limited tuning budget are consequently not standalone novelty claims of our design.

2.3 Agents, diagnosis, and accumulated knowledge

STELLAR already supports adaptive analysis, configuration execution, and accumulated rules with tuning contexts. Its rule synthesis also addresses conflicting guidance [5]. SysInsight derives hypotheses from code, induces contextual rules from observations, and maintains feedback-based reliability [21]. StorageXTuner, a preprint, combines agent roles, exploration, and empirically validated insights [12]. These mechanisms overlap substantially with a generic proposal for an agent with skills. We do not describe any of them as lacking context, feedback, or validation.

Our proposed distinction is an explicit action representation for *diagnostic interventions*: what can be changed or observed, which competing stage explanations it can distinguish, and what evidence is required before transferring its guidance. Whether this representation provides more than an application-specific reimplementation is an empirical question. A matched agent baseline and a matched rule-based controller are essential.

2.4 Measurement and comparison

Autotuning comparison must account for elapsed time, stochasticity, and evaluation overhead [20]. AutoTuneBench, a recent preprint, studies the reliability of agent-generated serving and kernel measurements [3]. We adopt independent metric computation and complete accounting as experimental requirements; these safeguards are not claimed as the primary novelty of this manuscript.

3 Historical Observations and Motivation

3.1 Data scope and metric semantics

The source is the author's 35-row CSV ledger for a model labelled `dsv4pro` and one named session workload. Row identifiers R1–R35 exclude the header. All records are marked PD-disaggregated: 34 describe 32-card 1P1D and R15 describes 48-card 2P1D. This is not a complete ratio sweep or a colocated comparison. The model label alone does not specify a reproducible checkpoint or precision. Image and branch columns are empty; version descriptions appear only in notes. Raw logs are referenced in the experimental environment but were not independently inspected for the present analysis.

The author confirms that the P and D columns report prefill and decode tokens per minute (TPM), total TPM includes cached tokens, and mean TPOT is in milliseconds. Prefill TPM accounts for 97.68–99.19% of total TPM in positive-throughput rows. Total TPM is thus an inclusive service measure, not newly executed prefill work or output generation rate. In R32, total TPM is 9.3565 million, whereas decode TPM is 131154.61, approximately 2185.91 tokens/s. Cache-layer attribution, weighting, and exact metric windows require the original script and logs.

Twenty-three rows report positive TPM, including degraded runs. Twelve contain zero TPM, including failed deployments, failed runs, and cache probes without complete metrics. Such zeros are not uniformly measured zero throughput, and zero latency fields must not be interpreted as instantaneous execution. The aggregates do not recover tail latency, joint SLO attainment, or independent-run uncertainty.

3.2 Cache context and resource trade-offs

Table 1 compares three historical 32-card 1P1D records at concurrency 88. R29 is labelled a cold single pass; R26 and R32 repeat the session workload for longer intervals. R29 to R32 shows a descriptive total-TPM difference of +45.29% and mean-TTFT difference of -75.74%, but mean TPOT increases by 21.53%. Cache state, duration, realized outputs, and request processes differ. This is not the effect of one optimization. An installed remote-cache backend also does not establish which tier supplied each hit.

Record/protocol	Cache hit (%)	Total TPM (10^6)	Mean TTFT (s)	Mean TPOT (ms)
R29: cold single pass	89.71	6.4401	14.9021	26.268
R26: 2 h repeated	98.43	9.2725	3.7549	32.431
R32: 24 h repeated	99.28	9.3565	3.6146	31.924

Table 1: Historical observations, not matched treatment effects. Protocols and cache states are ledger descriptions; raw-log validation remains necessary.

R11 and R15 illustrate another objective conflict. Moving from 32-card 1P1D to 48-card 2P1D is associated with +35.80% total TPM, -9.47% TPM per card, -66.17% mean TTFT, and +22.16% mean TPOT (Figure 1). Machine groups and runs differ. The points motivate separate resource and service objectives but establish neither a fixed-resource optimum nor a non-monotonic ratio curve.

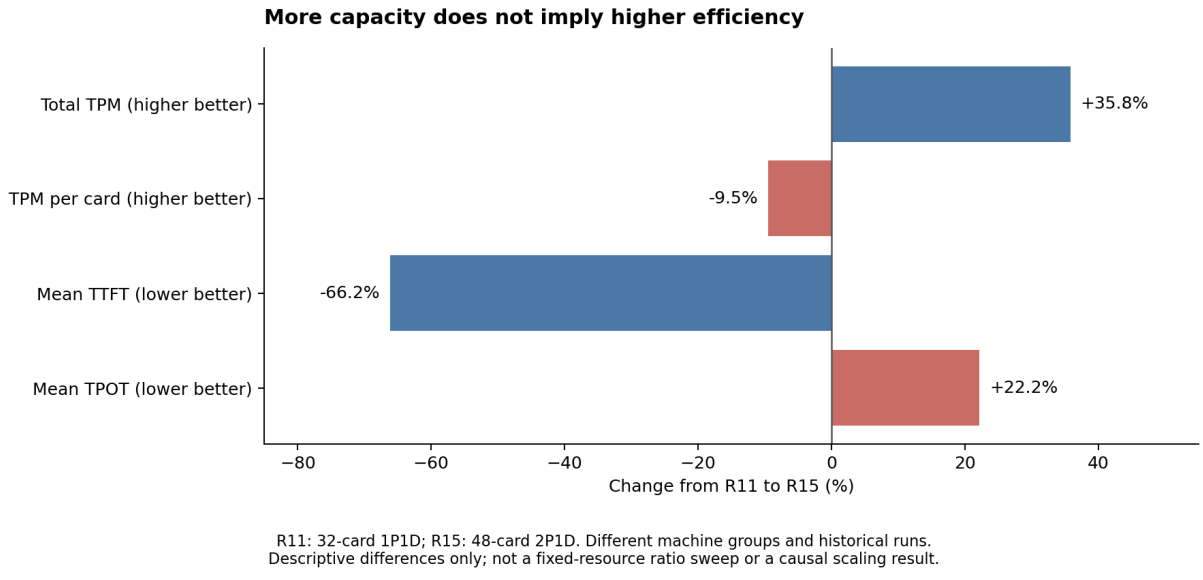


Figure 1: Descriptive changes from historical R11 to R15. More total throughput coexists with lower per-card throughput. Resource counts, machine groups, and uncontrolled conditions prevent a causal scaling interpretation.

3.3 Failure evidence and human decision provenance

The notes document changes in interpretation. R12 associates failure with `max_run96`; R13 reports restart-state incompatibility and retracts the earlier attribution; R14 still records instability after a complete restart. This motivates revisable explanations, not a permanent cross-version ban on a value. R16–R21 describe hanging, cache-loading, and KV-transfer symptoms under several interventions. They suggest competing explanations but do not independently prove unique root causes.

R19 reports a completed 1800-second run without a crash but only 992/14461 successful requests (6.86%). Process survival does not establish usable service. R32 reports 284733/284738 completions (99.998244%), including five failures. Its no-crash observation does not cover the idle-recovery path reported in R35 after a 5.5-hour idle interval. Repeating the same sessions is not independent workload generalization.

These diagnostic choices were made by the author, with skills executing selected operations. R34 documents a configuration, provisioning, pipeline, and reporting chain, with 1578/1606 completions (98.2565%). It is an execution foundation, not an autonomous-agent result. Historical cases may seed an authored library but must be separated from held-out evaluation. Root-cause claims and measurement-window reconciliation, including R19, require source logs before formal mechanism conclusions.

4 Problem Formulation

4.1 Tasks, configurations, and capabilities

A task is $\tau = (m, w, h, v, \rho, \Delta, \mathcal{S})$, where m identifies the model and fixed precision, w the request process, h the topology, v the software stack, ρ the initialization and state protocol, Δ the evaluation horizon, and $\mathcal{S}$ the service objectives. The protocol records cache clearing or reuse, deployment generations, idle exposure, session order, and warm-up boundaries. These are controlled conditions, not assumed observable internal state. Workload features include actual input/output length distributions and arrival variability. Maximum output length cannot replace the measured distribution.

A configuration is

$$c = (n_P, n_D, p_P, p_D, \pi, b_P, b_D, k), \tag{1}$$

where n_P, n_D are replica counts, p_P, p_D are supported parallel strategies, π maps workers to resources, b_P, b_D are runtime limits, and k contains supported state-transfer settings. The legal set $\mathcal{C}_\tau$ follows from the capability manifest, memory feasibility, and resource availability. Model and KV precision remain fixed within a comparison. Counts denote complete replicas, not arbitrary divisions of model weights.

Let $A(c)$ be the number of allocated accelerator cards; whole-node allocation is recorded separately when it determines reclaimable resources. We do not assume equal resource use per replica or that a model fits on a particular number of P800 cards. Structural changes can require redeployment. The first implementation is an offline planner; online elasticity and in-flight migration are outside its initial scope.

4.2 Service feasibility and objectives

For request i, define

$$z_i(c) = \mathbf{1}\{\text{complete}_i \wedge f_i \le \theta_f \wedge o_i \le \theta_o\}, \tag{2}$$

where f_i is client-observed TTFT and o_i is the request-level mean interval between output tokens. The primary workload will have at least two output tokens. Any shorter completions will be reported separately using a prespecified eligibility policy. Failures, timeouts, and rejected requests remain in the offered-request denominator.

For a fixed arrival cohort of N requests,

$$\widehat{a}(c) = \frac{1}{N} \sum_{i=1}^{N} z_i(c). \tag{3}$$

For a stable offered arrival rate λ, $\lambda \widehat{a}(c)$ is an attained-arrival-rate estimate. We will distinguish it from completion-window throughput and will report the arrival, observation, and drain intervals.

Queue stability and a fixed timeout policy are required to prevent short measurement windows from concealing overload. Separate marginal tail objectives do not imply the same joint attainment.

The capacity-planning objective is

$$\min_{c \in \mathcal{C}_\tau} A(c) \quad \text{subject to} \quad a(c) \geq \alpha, \quad \text{stable queues.} \tag{4}$$

The system will return the smallest *measured feasible* configuration it has found, not a certified global optimum. Within an equal-resource stratum, a secondary objective maximizes attained service capacity. A separate tuning objective minimizes the cost of reaching a prespecified feasible performance target. We will report these objectives separately rather than hiding resource consumption and latency in an arbitrary weighted score.

4.3 Budgeted evidence acquisition

At step t, the controller chooses an action a_t from reading existing evidence, executing a diagnostic skill, evaluating a configuration, or stopping. Its state is $s_t = (\tau, c_t, \mathcal{D}_t, \mathcal{K}_t, \mathcal{H}_t, B_t)$: observations, the skill library, revisable hypotheses, and remaining budget. $\mathcal{H}_t$ need not contain calibrated probabilities. A language model's confidence is not a measurement of bottleneck probability.

Every action has elapsed-time cost $T(a_t)$ and resource cost

$$C_A(a_t) = \int_{\text{action interval}} A_t(u)\, du. \tag{5}$$

Diagnosis, model loading, restarts, warm-up, recovery, and controller inference are charged. Concurrent experiments must satisfy a common resource ceiling or common allocation-time budget, with the selected protocol fixed across methods.

5 Proposed Method

5.1 Stage-aware evidence rather than a fixed latency equation

The controller uses a stage dependency graph containing client submission, request admission, prefill work, state readiness, transfer, decode admission, and token emission where those events are observable. This graph is a description of the deployed implementation, not a universally additive latency model. Transfer and computation can overlap. The origin and forwarding of the first streamed token are backend-dependent; decode admission or KV transfer must not automatically be charged to TTFT without tracing the actual first-token path.

Evidence records contain event semantics, clock domains, units, sample sizes, and missingness. Cross-node durations require synchronized clocks with characterized error or durations measured within one clock domain. The LLM receives references to these records and structured summaries. An unavailable counter is marked unknown rather than replaced by a plausible estimate. Diagnostic instrumentation is measured separately so that its overhead is not silently attributed to ordinary serving.

5.2 A diagnostic skill as an executable experiment

We define a skill by

$$\sigma = (P, O, H, I, V, F, R, E, G), \tag{6}$$

where P specifies applicability; O required observations; H explanations; I a bounded observation or intervention program; V outcomes of support, contradiction, or inconclusive; F parameter families; R restoration; E provenance-linked evidence; and G an externally checked constraint contract. Initial skills are engineering-authored. Historical notes supply candidate conditions and warnings, not automatically validated rules or causal labels.

A skill separates diagnostic outcomes from tuning rewards. A probe may support a transfer-contention hypothesis without improving end-to-end goodput. Conversely, a configuration may improve the objective while leaving its supposed explanation unresolved. Neither observation automatically validates the other. The record therefore has separate fields for hypothesis evidence, service feasibility, resource use, and objective change.

An initial library will cover prefill pressure, decode capacity or admission pressure, PD allocation imbalance, and transfer–compute interaction. These are alternative explanations and may coexist. Each skill must state what it holds fixed, what it varies, whether it needs redeployment, and what would make its outcome uninterpretable. A temporary change in cache state, precision, offered workload, or total allocation must not be presented as an isolated parameter effect.

5.3 Experience-constrained skill execution

The proposed contract separates four checks:

$$G = (G_{\text{cap}}, G_{\text{exec}}, G_{\text{eval}}, G_{\text{reuse}}). \tag{7}$$

Capability checks restrict operations to the deployed manifest. Execution checks require typed parameters, resource reservation, declared intervention scope, restoration, and budget bounds. Evaluation checks preserve model/precision, workload protocol, metric formulas, and SLO denominator. Reuse checks bind guidance to a versioned state context and request observations or replay when needed. Missing preconditions produce explicit unresolved or non-executable outcomes; the LLM must not silently fill them.

Contract	Historical motivation	Intended executor behavior
Capability	Incomplete version descriptions	Reject unsupported operations; record unknown effects
Execution/state	R12–R14 restart ambiguity	Check worker generations; use a declared recovery protocol
Evaluation	R19 survives but completes few requests	Separate survival, completion, and joint SLO attainment
Reuse	R32 long run versus R35 idle recovery	Reassess cache, idle, version, and topology changes

Table 2: Proposed skill constraints motivated by historical problems, not a validated controller.

All controllers receive the same mandatory legality and measurement checks. The fixed-rule baseline receives identical skills and initial evidence. Otherwise gains might reflect unequal safeguards rather than agent decisions. Historical runtime-limit warnings remain context-local guidance; only independently established capabilities justify hard exclusion from the legal space.

Restart consistency illustrates the distinction. The study can require a coordinated P/D restart for a declared transition, while a backend-certified compatible partial restart remains legal under its manifest. One historical failure is not a universal restart rule. Disabling asynchronous loading is likewise not accepted as an improvement merely because a crash disappears: service feasibility and cost are evaluated separately.

Each invocation records contract decisions, input/context hashes, source runs, expected distinguishing observations, outcome, objective change, restoration, and charged cost. Historical evidence is tagged human-directed and log-unverified until checked. Later contradictions are appended with scope and reason rather than rewriting observations. The JSON artifact specifies contracts but has no executable backend bindings.

5.4 Applicability checks and local replay

Reuse proceeds through three checks. First, an executable compatibility predicate checks the observations and actions needed by the skill. An absent capability rejects the action. A software or

model change whose effects are unknown instead makes the skill unvalidated for the new context; it need not make the diagnostic procedure itself illegal.

Second, the controller compares the current context with contexts in the evidence record. Workload and scale features are normalized using a transformation fixed on source tasks. A transparent distance and coverage rule will flag extrapolation. Thresholds will be chosen on source validation tasks and frozen before held-out evaluation. Similarity is only a retrieval aid and is not a correctness guarantee.

Third, stale, insufficient, or out-of-coverage guidance requires local replay of its diagnostic action. A replay can revalidate an expectation, contradict it, or remain inconclusive. The last outcome does not count as a negative result. Evidence is stored by task and context so that failure on one topology does not erase valid observations elsewhere. Revalidation reduces unsupported transfer; it provides no universal guarantee that a skill is useful.

5.5 Selecting a diagnostic action

The LLM proposes a bounded list of skill identifiers and attaches evidence references, competing explanations, and predicted distinguishing observations. It produces structured decisions, not an unrestricted script or a self-certified diagnosis. A deterministic checker verifies that each proposal uses available observations and a legal intervention. Duplicate or non-discriminating proposals are returned for revision.

The initial selection policy will prefer reading already collected evidence, followed by a low-cost diagnostic expected to separate unresolved explanations. An experimental candidate score is

$$Q(d) = \frac{U(d)}{\widehat{C}(d) + \varepsilon}, \tag{8}$$

where $U(d)$ estimates discrimination among current explanations and $\widehat{C}(d)$ estimates cost under the selected accounting protocol. Before a calibrated predictor exists, $U(d)$ will be an explicit ordinal engineering rubric with an "unknown" value; it will not be advertised as expected information gain or calibrated causal confidence. The rubric, cost estimator, and tie-breaking policy will be fixed before comparison. The value of LLM proposal generation will be evaluated against an identical rule-based selector.

The diagnostic evidence updates a set of supported, contradicted, and unresolved expectations. Multiple bottlenecks are allowed. When the available probes cannot discriminate, the controller falls back to broader numerical exploration. This fallback is important: the design does not assume that every workload admits an inexpensive and correct diagnosis.

5.6 Guiding numerical search without permanent exclusion

A supported diagnostic prioritizes a parameter family and suggests legal initialization points. Structural proposals include replica counts and worker placement; within a structure, the numerical optimizer chooses supported runtime values. The first implementation will use one documented constrained optimizer and retain an unmodified version as a baseline.

Skill guidance affects proposal priority, not hard feasibility, except for independently established capability constraints. In particular, a fraction $\epsilon > 0$ of proposal opportunities remains available to the full legal space:

$$q_t(c) = (1 - \epsilon) q_t^{\text{guided}}(c) + \epsilon q_t^{\text{global}}(c). \tag{9}$$

The proposal distributions and ϵ will be declared in the implementation specification. Full-space sampling does not prove convergence under a finite budget; its purpose is to limit irreversible exclusion caused by an incorrect diagnosis. A fixed no-improvement rule can trigger a return to diagnosis or increase global exploration, with its extra cost recorded.

An accepted configuration must be measured. A predictor or an agent-generated explanation cannot make an untested configuration the final recommendation. Final evaluation uses a frozen

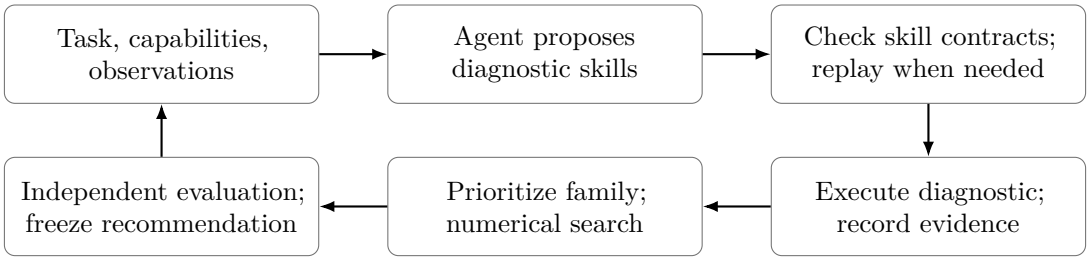


Figure 2: Proposed control loop. The diagram specifies a design, not a completed implementation. Updates to skill evidence retain raw run provenance.

configuration and independent requests; exploratory measurements do not substitute for held-out service validation.

5.7 Updating evidence and stopping

Skill records preserve positive, negative, and inconclusive outcomes, context signatures, configuration changes, and raw run identifiers. Empirical support frequencies, if displayed, are conditional on observed interventions and are not interpreted as causal probabilities. The first version avoids learning an unrestricted causal model from a few tuning runs.

Stopping occurs on budget exhaustion, attainment of a prespecified target followed by independent confirmation, or a declared termination rule. An agent may request stopping, but the executor checks the rule. If no feasible configuration is measured, the output reports that outcome and the tested scope. It does not silently relax the SLO or change the model.

6 Integration Plan and Algorithm

6.1 SGLang and P800 adapter boundary

The adapter will expose capability discovery, deployment, observation collection, diagnostic execution, benchmark execution, restoration, and configuration freezing. Logical parameters are translated to flags only after checking the deployed version. The controller operates outside the token-generation hot path. It is not proposed as a per-token scheduler.

The capability manifest records model and tokenizer identifiers, weight and KV types, runtime and plugin revisions, attention implementation, parallel configurations, graph and cache modes, transfer backend, and supported routing. CPU, XPU, PCIe, and NIC topology are inventoried instead of inferred from socket count. Unknown single-card memory, NIC speed, and transport behavior remain unset until measurement. Published plugin support cannot replace acceptance checks on the local image.

Deployment operations carry a resource reservation and restoration plan. Diagnostic probes run on designated experimental resources, and all benchmark parsing and objective computation occur outside agent-editable decisions. The agent can select registered operations, but cannot modify evaluation inputs, metric formulas, or baseline configurations during a comparison. This is an experimental control rather than a separate security contribution.

6.2 Proposed algorithm

```
Input:  task, legal resources, capability manifest, initial skills, budgets.
1.  Measure a declared initial configuration; create evidence records.
2.  While budget remains:
  a.  Retrieve compatible skill candidates and context records.
  b.  Request structured diagnostic proposals from the controller.
  c.  Check contracts and state; read or replay diagnostics.
  d.  Record evidence, provenance, and service outcomes.
  e.  Propose a legal configuration with guided/global search.
  f.  Deploy, warm, measure, restore when required; charge all costs.
  g.  Update the optimizer and context-specific skill records.
  h.  Check the declared stop or return-to-diagnosis rule.
3.  Freeze and independently validate the best measured feasible candidate.
Output:  configuration and evidence, or no measured feasible configuration.
```

This is the intended autonomous sequencing. Existing skills execute human-selected tasks; the complete agent loop, checker, optimizer settings, context thresholds, and stopping rules still require integration and evaluation. Historical tool execution does not establish these proposed components as implemented.

7 Planned Evaluation

7.1 Research questions

RQ1 asks whether the method reaches a prespecified feasible target faster than constrained numerical search and an ordinary tool-using agent. RQ2 asks whether diagnostic selection changes useful configuration decisions relative to a fixed-rule controller with identical skills. RQ3 asks whether local replay and applicability checks improve skill transfer to held-out contexts. RQ4 asks how the final recommendations compare in resource use and attained service capacity across deployment scales.

7.2 Testbed, tasks, and splits

The starting testbed comprises four eight-P800 servers with dual-socket AMD EPYC 9K84 CPUs; the author reports a 96-core specification and RDMA support. Historical R15 used 48 cards in another group. The per-socket interpretation, NUMA layout, memory, NIC configuration, and exact revisions will be inventoried. The pool is expandable; no unmeasured larger layout is assumed valid.

We plan to pin the current checkpoint and precision and, where feasible, add a second stable model configuration. Historical notes describe a 16-card prefill replica (PP2, TP8 per stage) and a 16-card decode replica. If verified and supported, comparisons include 2P1D versus 1P2D at 48 cards and 3P1D/2P2D/1P3D at 64 cards, alongside 32-card 1P1D. These are planned layouts, not results. Unsupported layouts are marked infeasible without changing precision or replacing the model inside a comparison.

Tasks will include short-input/short-output, long-input/short-output, short-input/long-output, and mixed-length request processes. Open-loop arrivals will be used for service-capacity studies, with fixed-concurrency tests as secondary diagnostics. Source tasks build the skill evidence. Context thresholds and the initial library are then frozen before held-out model, load, or scale tasks. Cold-start, warm-start, and optional within-task updates are reported separately.

The 35 historical records are development evidence, not a final test set. Human-directed choices initialize authored skills but cannot establish autonomous selection accuracy. Before prospective runs, freeze source/validation/test splits, equal initial knowledge, prompts and optimizer settings, and cache/restart/idle protocols. Recover traces and manifests for key cases before repeating unresolved measurements. Continuous-load and idle-recovery tests remain separate conditions.

7.3 Baselines and component attribution

Method	Shared conditions	Purpose
Tuned native serving	Colocated/chunked-prefill and native PD; fixed model and precision	End-to-end engineering reference
Constrained optimizer	Same legal space, budgets, and measurement protocol	Benefit beyond numerical search
Fixed-rule controller	Same skills, observations, and initial knowledge	Necessity of LLM decisions
Tool-using agent	Same model and tools; ordinary feedback and log retrieval	Benefit beyond generic agent orchestration
Unconditional skill reuse	Same library; no guidance revalidation; mandatory checks retained	Cost of incorrect transfer
Complete proposal	All declared components	Combined effect and overhead

Table 3: Planned comparisons. Adapting ideas from prior systems must be identified as an adaptation, not as running the original implementation.

A source-aware or rule-accumulating agent inspired by STELLAR or SysInsight is an important strong baseline. If their original implementations cannot control SGLang, an adaptation will be described precisely and supplied with the same evidence and tools. We will not reuse reported speedups from unrelated hardware as quantitative baselines. Ablations will remove active diagnosis, replace skill records with raw-log retrieval, disable applicability replay, and remove global exploration, one mechanism at a time.

7.4 Metrics, uncertainty, and resource accounting

Primary metrics are allocation-time and elapsed time to a prespecified target, joint SLO attainment, and final measured resource use at the same offered workload. Secondary metrics include output-token throughput, actual input/output lengths, per-request TTFT/TPOT, token-level inter-token latency, failed configurations, redeployments, and diagnostic actions. P99 results require adequate sample sizes and explicit uncertainty. Diagnostic precision will be reported only for cases with controlled or independently established explanations.

We plan at least three independent repeats of core serving points and multiple controller/search seeds, initially five where affordable. Run order will be randomized or interleaved. Paired comparisons will use the same workload realizations. Requests within one overloaded run are not treated as independent repetitions of the entire tuning process. Unreached targets will be reported as censored outcomes, not dropped when calculating time-to-target.

Thresholds, model selection, search ranges, stopping conditions, the agent prompt, generation settings, and token budgets will be fixed before final evaluation. Agent inference will use separate capacity or have its interference explicitly measured. Parallel searches will share an equal resource ceiling or be compared using an equal allocation-time budget. Offline library construction and online tuning costs will be separated, with amortization reported rather than assumed.

7.5 Mechanism tests and decision consequences

The first mechanism test compares isolated transfer, isolated decode, and their controlled concurrent execution when the backend exposes suitable operations. Another compares a stage-capacity recommendation with end-to-end configuration ranking. These tests establish whether the hypothesized ambiguity exists on this platform; they do not automatically establish a novel system mechanism.

For each successful diagnostic, we will trace the decision chain: observations, alternatives considered, chosen probe, probe outcome, changed search priority, selected configuration, and

independent validation. For an incorrect diagnostic, we will report the wasted cost and whether replay or global exploration recovered. This analysis connects the skill mechanism to a measurable outcome rather than treating explanation quality as a proxy for performance.

8 Prospective Agent Results and Required Evidence

The historical observations in Section 3 are available; matched autonomous-controller results are not. This section specifies evidence needed to assess the proposed method, rather than attributing human-directed outcomes to an agent. The main results will address RQ1–RQ4 with paired uncertainty estimates. Required figures are: time-to-target and attained-quality-over-budget curves; resource–service-capacity frontiers; diagnostic outcomes and their decision consequences; held-out skill transfer with and without replay; and overhead/failed-action breakdowns. No synthetic speedup plot is included.

Evidence	Required analysis	Current status
E1: mechanism	Competing explanations and controlled diagnostic outcomes	Not measured
E2: tuning	Same-budget baselines and time-to-target	Not measured
E3: transfer	Frozen source library on held-out contexts	Not measured
E4: capacity	Same-load resource use and equal-resource service ability	Not measured
E5: ablation	Rules, ordinary agent, replay, and exploration controls	Not measured

Table 4: Evidence completion ledger. These entries are status labels, not experimental data.

Negative findings must change the manuscript's claim. If rules match the LLM controller, we cannot attribute a performance advantage to agent reasoning. If replay costs more than it saves, reuse requires a narrower applicability argument. If no meaningful stage ambiguity appears, the proposed diagnostic design needs revision. A gain over defaults alone will not establish a method contribution.

9 Limitations and Threats to Validity

The main limitation is the absence of a validated autonomous integration and matched method evaluation. Historical evidence is aggregate, heterogeneous, and human-directed; missing version manifests and raw-log verification limit mechanism conclusions. The distinction from agentic storage and database tuners remains a hypothesis. Existing systems already handle context and conflicts, so constrained diagnostic actions must show value beyond renaming rules as skills.

A single accelerator backend limits hardware generalization. Larger resource counts extend scale coverage but do not establish portability to a different accelerator family. Limited observability may prevent reliable diagnosis, while diagnostic instrumentation can perturb execution. Context signatures and thresholds may overfit source tasks. Dynamic co-tenancy, request-length uncertainty, cache state, and network variation may confound intervention outcomes even when only one configuration field is edited.

An LLM controller can omit important alternatives, fabricate parameter semantics, or select expensive probes. Registered actions, evidence references, and independent checking constrain these behaviors but do not prove their absence. Improved tuning time also does not imply better steady-state capacity. Conversely, a more efficient final deployment may require greater tuning cost; both must be reported.

The initial scope excludes automatic kernel rewriting, model-quality tradeoffs, cross-vendor KV interoperability, and production online migration. Public models and reproducible workload

descriptions will be preferred; disclosure of actual code, machine details, and traces depends on the organization's permitted publication scope. Unavailable artifacts will be listed explicitly rather than described as released.

10 Conclusion

This manuscript connects 35 historical P800 experiment records with an experience-constrained diagnostic-skill design for SGLang PD serving. The records motivate context-aware evaluation, revisable explanations, and separate survival, completion, and SLO outcomes. The central hypothesis is that an agent can use these bounded skills and locally revalidated experience to improve configuration decisions at an acceptable total tuning cost. Autonomous integration and matched experiments are required before claiming a method advantage.

A Implementation Decisions That Must Be Frozen

Before final experiments, the artifact must specify: the model and precision identifiers; SGLang, plugin, and transport revisions; the mapping from logical actions to supported operations; controller model and prompts; search algorithm and settings; skill contexts and replay thresholds; cost-accounting protocol; arrival and drain semantics; SLOs and timeouts; source/validation/test splits; random seeds; and the conditions for stopping. These are open implementation decisions, not omitted experimental findings.

B Claim-to-Evidence Map

Candidate claim	Minimum supporting evidence
Diagnostic selection improves tuning decisions	Same-tool rule and agent controls; decisions linked to held-out performance
Applicability replay reduces negative transfer	Frozen source library; changed-context tasks; unconditional-reuse and cost ablations
Resource recommendations are efficient	Same workload/SLO; measured feasible resource comparison; stability and uncertainty
The method is practical on P800	Versioned integrated artifact; acceptance checks; complete overhead and failure records

C Historical Data and Decision Provenance

The ledger is stored as `experiments/dsv4pro_32card_pd_ratio_scan.csv`. Its SHA-256 is

`e688fa3dabfea4dab60d4fab8eb32b69c0cd0f551adb1ca9344036602429dfaa`.

R1–R35 are one-based data rows. The analysis script preserves source values and adds row identifiers, availability flags, and arithmetic checks; it does not turn missing zeros into successful observations. The author confirmed TPM semantics, TPOT units, and human diagnostic selection. The CSV references source logs, but the present analysis uses the ledger rather than a raw-log audit. Reported fixes and fault attributions are historical notes, not claimed repair algorithms or verified causal labels. Subsequent agent runs require new identifiers and decision provenance; they must not relabel these records as autonomous trials.

References

[1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation*, pages 117–134. USENIX Association, 2024. URL https://www.usenix.org/conference/osdi24/presentation/agrawal.

[2] Baidu Baige Contributors. SGLang-Kunlun: Hardware platform plugin. Project repository, 2026. URL https://github.com/baidu-baige/sglang-kunlun. Accessed 2026-10-18; deployed version not yet supplied.

[3] Li Chen. AutoTuneBench: Trustworthy measurement for agent auto-tuning of LLM serving engines. arXiv:2609.18123, 2026. URL https://arxiv.org/abs/2609.18123. Preprint.

[4] Ke Cheng, Zhi Wang, Wen Hu, Tiannuo Yang, Jianguo Li, and Sheng Zhang. SCOOT: SLO-oriented performance tuning for LLM inference engines. In *Proceedings of the ACM Web Conference 2025*, pages 829–839, 2025. doi: 10.1145/3696410.3714930. URL https://arxiv.org/html/2408.04323v2.

[5] Chris Egersdoerfer, Philip Carns, Shane Snyder, Robert Ross, and Dong Dai. STELLAR: Storage tuning engine leveraging LLM autonomous reasoning for high performance parallel file systems. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025. doi: 10.1145/3712285.3759887. URL https://arxiv.org/html/2602.23220v1.

[6] Victor Giannakouris and Immanuel Trummer. λ-tune: Harnessing large language models for automated database system tuning. *Proceedings of the ACM on Management of Data*, 3(1), 2025. doi: 10.1145/3709652. URL https://2025.sigmodconf.hosting.acm.org/toc-3-1.html.

[7] Youhe Jiang, Ran Yan, and Binhang Yuan. HexGen-2: Disaggregated generative inference of LLMs in heterogeneous environment. In *International Conference on Learning Representations*, 2025. URL https://openreview.net/pdf?id=Cs6MrbFuMq.

[8] Ruiqi Lai, Hongrui Liu, Chengzhi Lu, Zonghao Liu, Siyu Cao, Siyang Shao, Yixin Zhang, Luo Mai, and Dmitrii Ustiugov. TokenScale: Timely and accurate autoscaling for disaggregated LLM serving with token velocity. arXiv:2512.03416, 2025. URL https://arxiv.org/abs/2512.03416. Preprint.

[9] Jiale Lao, Yibo Wang, Yufei Li, Jianping Wang, Yunjia Zhang, Zhiyuan Cheng, Wanghu Chen, Mingjie Tang, and Jianguo Wang. GPTuner: A manual-reading database tuning system via GPT-guided bayesian optimization. *Proceedings of the VLDB Endowment*, 17(8):1939–1952, 2024. doi: 10.14778/3659437.3659449. URL https://vldb.org/pvldb/vol17/p1939-tang.pdf.

[10] Jiaxi Li, Yue Zhu, Eun Kyung Lee, and Klara Nahrstedt. Revisiting disaggregated large language model serving for performance and energy implications. arXiv:2601.08833, 2026. URL https://arxiv.org/abs/2601.08833. Technical-report version supplied in the research materials.

[11] Luchang Li, Dongfang Li, Bozhao Gong, and Yu Zhang. SLO-aware compute resource allocation for prefill-decode disaggregated LLM inference. arXiv:2603.04716, 2026. URL https://arxiv.org/abs/2603.04716. Preprint.

[12] Qi Lin, Zhenyu Zhang, Viraj Thakkar, Zhenjie Sun, Mai Zheng, and Zhichao Cao. StorageXTuner: An LLM agent-driven automatic tuning framework for heterogeneous storage systems. arXiv:2510.25017, 2025. URL https://arxiv.org/abs/2510.25017. Preprint.

[13] Christian Lysenstøen. SLO-Guard: Crash-aware, budget-consistent autotuning for SLO-constrained LLM serving. arXiv:2604.17627, 2026. URL https://arxiv.org/abs/2604.17627. Preprint.

[14] Meta Inference Team. Optimizing deployment configurations for LLM inference: Challenges and insights. In *Proceedings of the 9th MLSys Conference, Industry Track*, 2026. URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/97dc07f1253ab33ee514f395a82fa7cc-Abstract-Conference.html.

[15] Tiyasa Mitra, Ritika Borkar, Nidhi Bhatia, et al. A pragmatic exploration of prefill-decode disaggregation in large scale inference. In *Proceedings of the 9th MLSys Conference, Industry Track*, 2026. URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/d49cee5f3a79d97d719df255689d83d7-Abstract-Conference.html.

[16] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *ACM/IEEE International Symposium on Computer Architecture*, 2024. URL https://www.microsoft.com/en-us/research/publication/splitwise-efficient-generative-llm-inference-using-phase-splitting/.

[17] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. arXiv:2407.00079, 2024. URL https://arxiv.org/abs/2407.00079. Cited version is the technical report, not the later journal version.

[18] SGLang Contributors. PD disaggregation. SGLang documentation, 2026. URL https://docs.sglang.io/docs/advanced_features/pd_disaggregation. Accessed 2026-10-18; a release or commit must be pinned for experiments.

[19] Chao Wang, Pengfei Zuo, Zhangyu Chen, Yunkai Liang, Zhou Yu, and Ming-Chang Yang. Prefill-decode aggregation or disaggregation? unifying both for goodput-optimized LLM serving. arXiv:2508.01989, 2025. URL https://arxiv.org/abs/2508.01989. Cited technical-report version.

[20] Floris-Jan Willemsen, Richard Schoonhoven, Jiří Filipovič, Jacob O. Tørring, Rob van Nieuwpoort, and Ben van Werkhoven. A methodology for comparing optimization algorithms for auto-tuning. *Future Generation Computer Systems*, 159:489–504, 2024. doi: 10.1016/j.future.2024.05.021. URL https://ac.liacs.nl/blog/methodology-paper.

[21] Xinyi Zhang, Tiantian Chen, Zhentao Han, Zhaoyan Hong, Wei Lu, Sheng Wang, Mo Sha, Anni Wang, Shuang Liu, Yakun Zhang, Feifei Li, and Xiaoyong Du. Why database manuals are not enough: Efficient and reliable configuration tuning for DBMSs via code-driven LLM agents. *Proceedings of the VLDB Endowment*, 19(6):1358–1371, 2026. doi: 10.14778/3797919.3797940. URL https://arxiv.org/html/2603.22708v1.

[22] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.nips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html.

[23] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation*, pages 193–210. USENIX Association, 2024. URL https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin.

[24] Qingyun Zou, Bin Gao, Zhuobin Huang, Weng-Fai Wong, Tulika Mitra, and Bingsheng He. ECO: Energy-oriented configuration optimization for attention FFN disaggregated LLM serving. arXiv:2610.08373, 2026. URL https://arxiv.org/abs/2610.08373. Preprint.